

Network Simulator

Table of Contents

- [Packet](#)
- [Jitter Buffer](#)
- [Network Simulator](#)
- [VOIP Sender](#)
- [VOIP Receiver](#)
- [Audio Capture](#)

Packet

class packet.**Packet** [\[source\]](#)

Bases: **object**

HEADER_FORMAT = 'i i i 16s'

HEADER_SIZE = 28

MAX_PAYLOAD = 1024

checksum() [\[source\]](#)

Creates md5 digest which can be used to detect if bits are corrupted

params:

data: current set of data being transferred

create_packet(ack_num, flags, payload=b'') [\[source\]](#)

Creates packets that are transferred between the client and server

params:

seq_num: current sequence number acl_num: current ACK number flags: flags triggered during transfer

payload: data being sent in current packet

unpack() [\[source\]](#)

unpacks raw bytes being transferred

params:

raw_bytes: raw bytes of data currently in packet

Jitter Buffer

class jitterbuffer.**JitterBuffer**(max_size=250) [\[source\]](#)

Bases: **object**

Buffer built for the VOIP connectio, ignores small errors instead of resending packets

add_packet(seq_num, payload) [\[source\]](#)

adds audio data to the jitter buffer

params:

seq_num: number of the packet in sequence payload: the audio data itself

get_next_packet()

[\[source\]](#)

returns the next packet in the buffer

Network Simulator

`network_sim.network_sim(client_port, target_ip, target_port, loss, corrupt, reorder)` [\[source\]](#)

Simulates a network connection between a client and server. Includes data loss check and corrupted data check

params:

client_port: port of the connected client target_ip: ip of the of the target server target_port: port of the target server loss: chance for a simulated data loss to be triggerd corrupt: chance for a simulated data corruption to occur reorder: chance for a simulated reorder to occur

VOIP Sender

`class voip_send.VOIP_Sender(target_ip, target_port)` [\[source\]](#)

Bases: **object**

Object that starts VOIP connection

save_audio(filename='sent_live_audio.wav', sample_rate=8000) [\[source\]](#)

Saves the audio data and prints it to a .wav file to listen to

params:

filename: name of the file audio is being saved to sample_rate: sample rate of audio

send_audio(audio_data) [\[source\]](#)

Sends audio data to receiver

params:

audio data: Sound data to be sent to the receiver connection

VOIP Receiver

`class voip_receive.VOIP_Receiver(listen_ip, listen_port)` [\[source\]](#)

Bases: **object**

Object that receives the VOIP connection

listen() [\[source\]](#)

listend for a connection request from the sender

play_audio() [\[source\]](#)

plays the live audio being sent from the sender

receive_audio() [\[source\]](#)

starts listen and play functions as threads

save_audio(filename='received_live_audio.wav', sample_rate=8000) [\[source\]](#)

Saves the audio data and prints it to a .wav file to listen to

params:

filename: name of the file audio is being saved to sample_rate: sample rate of audio

Audio Capture

```
rec_sound.live_capture(sample_rate, filename='client_live_output.wav')  
rec_sound.record_audio(duration, sample_rate,  
filename='output.wav')
```

[source]

[source]